

ОТЧЕТ ПО HW4

HW4: MyDrive

ФИО: Гареев Данир Фанзилович

Группа: БПИ-236

Email: dfgareev@edu.hse.ru

Я претендую на оценку до 10 баллов, сниженной коэффициентом 0.3 из-за сдачи после дедлайна.

Цель работы

Целью работы является разработка многопользовательского клиент-серверного приложения для синхронизации файловой директории по протоколу TCP, а также экспериментальное исследование поведения TCP при передаче файлов большого размера, в том числе в режимах обычной передачи и передачи с использованием DMA/zero-copy.

Выбранные язык и framework

Для реализации решения были выбраны язык программирования C++ и сетевой framework **Boost.Asio**. Такой выбор соответствует требованиям задания и позволяет реализовать всю сетевую логику на уровне низкоуровневого TCP-взаимодействия без использования coroutines.

Архитектура решения

Решение состоит из двух отдельных консольных приложений:

- `mydrive_server` — асинхронный TCP-сервер, обслуживающий управляющие и файловые соединения клиентов.
- `mydrive_client` — клиент синхронизации, сканирующий локальную директорию и отправляющий на сервер только отсутствующие или измененные файлы.
- `common` — общая библиотека с протоколом сообщений, сериализацией, checksum, конфигами и файловыми утилитами.

На сервере файлы хранятся в структуре `storage_root/client_id/...`. Для каждого пользователя используется отдельная поддиректория, что позволяет обслуживать множество клиентов независимо друг от друга.

Управляющий обмен и передача содержимого файлов разделены:

- **control connection** используется для сообщений `Hello`, `FileList`, `SyncPlan`, `SyncComplete`.
- **file transfer connections** используются для параллельной отправки файлов.

Бизнес-протокол и communication pattern

Поверх TCP реализован собственный бинарный протокол с явным фреймингом:

- `magic`
- `protocol_version`
- `message_type`
- `payload_length`
- `payload`

Такой формат позволяет корректно собирать сообщения из частичных TCP-чтений и затем выполнять десериализацию уже после полного накопления полезной нагрузки.

В решении используются следующие типы сообщений:

- Hello { protocol_version, client_id }
- FileList { files[] }
- SyncPlan { files_to_upload[] }
- TransferRequest { transfer_id, client_id, relative_path, size, checksum, use_dma }
- TransferResult { transfer_id, status, bytes_received, details }
- SyncComplete { ok, details }
- Error { details }

Информационные сообщения сериализуются в массив байт и явно передаются по TCP. Содержимое файла передается отдельно, бинарным потоком байт по выделенному файловому соединению, а не как сериализованный объект.

Ключевые фрагменты реализации

Серверная часть

Сервер принимает неограниченное число входящих TCP-соединений. После чтения первого фрейма он определяет тип соединения:

- если первым сообщением является Hello, соединение переводится в ControlSession;
- если первым сообщением является TransferRequest, соединение переводится в TransferSession.

ControlSession отвечает за построение плана синхронизации, а TransferSession принимает поток байт файла, записывает его во временный .part-файл, проверяет checksum и затем выполняет атомарную замену целевого файла.

Клиентская часть

Клиент при запуске:

- загружает конфигурацию;
- сканирует верхний уровень заданной директории;
- вычисляет размер и checksum каждого файла;
- отправляет серверу Hello и FileList;
- получает SyncPlan;
- запускает до max_connections параллельных upload-соединений;
- после завершения всех upload-ов отправляет SyncComplete.

Надежность и обработка corner cases

В реализации предусмотрены следующие меры надежности:

- запись во временный .part-файл;
- атомарное переименование файла после успешной верификации;
- проверка checksum после приема;
- запрет path traversal через .. и абсолютные пути;
- ограничение размера управляющего payload;
- отказ от загрузки файла целиком в память.

Экспериментальная установка

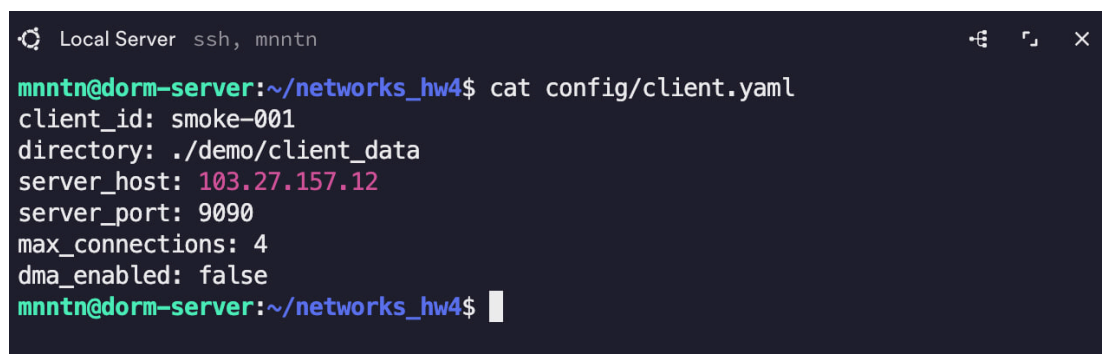
Параметр	Значение
Язык	C++
Framework	Boost.Asio
IP сервера	103.27.157.12
ОС сервера	Удаленная Linux VM (Ubuntu-сервер)
ОС клиента	Отдельная Linux-машина для запусков и замеров; Wireshark использовался на macOS для анализа .pcap
Сборка	сmake -S . -B build и смake --build build -j
Инструменты наблюдения	ss, tcpdump, strace, Wireshark

Проверка работоспособности приложения

Были выполнены несколько последовательных сценариев проверки: первичная синхронизация небольших файлов, повторная синхронизация без изменений, инкрементальная синхронизация после изменения содержимого и добавления нового файла, параллельная передача четырех больших файлов, а также серия замеров времени для режимов DMA off и DMA on.

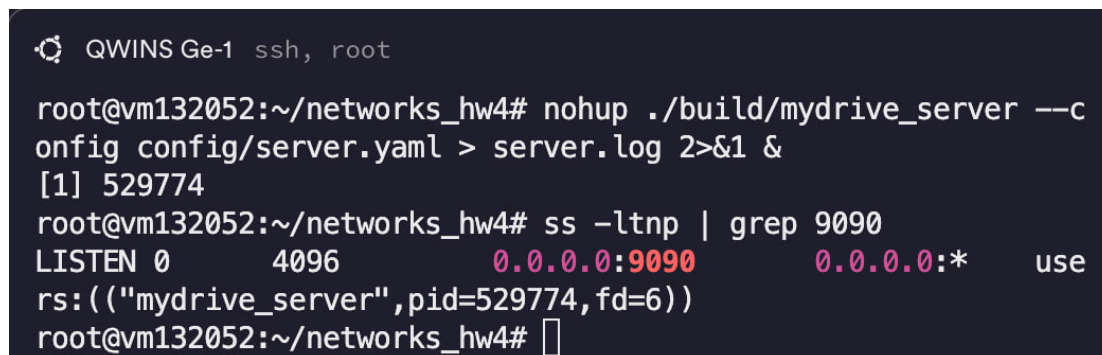
Логи и скриншоты запуска

Ниже приведены скриншоты основных базовых сценариев.



```
Local Server ssh, mnntn
mnntn@dorm-server:~/networks_hw4$ cat config/client.yaml
client_id: smoke-001
directory: ./demo/client_data
server_host: 103.27.157.12
server_port: 9090
max_connections: 4
dma_enabled: false
mnntn@dorm-server:~/networks_hw4$
```

Рис. 1. Конфигурация клиента для первого теста: client_id, директория синхронизации, адрес и порт сервера.



```
QWINS Ge-1 ssh, root
root@vm132052:~/networks_hw4# nohup ./build/mydrive_server --c
onfig config/server.yaml > server.log 2>&1 &
[1] 529774
root@vm132052:~/networks_hw4# ss -ltnp | grep 9090
LISTEN 0      4096      0.0.0.0:9090  0.0.0.0:*    use
rs: ("mydrive_server",pid=529774,fd=6)
root@vm132052:~/networks_hw4#
```

Рис. 2. Запуск серверной части и проверка того, что сервер слушает TCP-порт 9090.

```

mnntn@dorm-server:~/networks_hw4$ printf 'alpha\n' > demo/client_data/a.txt
printf 'beta\n' > demo/client_data/b.txt
printf 'gamma\n' > demo/client_data/c.txt
ls -lh demo/client_data
time ./build/mydrive_client --config config/client.yaml sync
total 12K
-rw-rw-r-- 1 mnntn mnntn 6 мая 20 19:52 a.txt
-rw-rw-r-- 1 mnntn mnntn 5 мая 20 19:52 b.txt
-rw-rw-r-- 1 mnntn mnntn 6 мая 20 19:52 c.txt
Scanning "./demo/client_data"
upload b.txt: ok (0.422s) Upload complete
upload c.txt: ok (0.48s) Upload complete
upload a.txt: ok (0.483s) Upload complete

real    0m0,927s
user    0m0,002s
sys     0m0,001s
mnntn@dorm-server:~/networks_hw4$

```

Рис. 3. Первая синхронизация набора небольших файлов с клиента на сервер.

```

root@vm132052:~/networks_hw4# find demo/server_storage -maxdep
th 2 -type f
ls -lh demo/server_storage/smoke-001
demo/server_storage/smoke-001/c.txt
demo/server_storage/smoke-001/a.txt
demo/server_storage/smoke-001/b.txt
total 12K
-rw-r--r-- 1 root root 6 May 20 19:52 a.txt
-rw-r--r-- 1 root root 5 May 20 19:52 b.txt
-rw-r--r-- 1 root root 6 May 20 19:52 c.txt
root@vm132052:~/networks_hw4#

```

Рис. 4. На сервере появились три синхронизированных файла после первого теста.

```

mnntn@dorm-server:~/networks_hw4$ time ./build/mydrive_client --config config
/client.yaml sync
Scanning "./demo/client_data"

real    0m0,539s
user    0m0,002s
sys     0m0,000s
mnntn@dorm-server:~/networks_hw4$

```

Рис. 5. Повторная синхронизация без изменений: дополнительных upload-ов нет.

```

mnntn@dorm-server:~/networks_hw4$ printf 'alpha\nchanged\n' > demo/client_data/a.txt
printf 'delta\n' > demo/client_data/d.txt
ls -lh demo/client_data
time ./build/mydrive_client --config config/client.yaml sync
total 16K
-rw-rw-r-- 1 mnntn mnntn 14 мая 20 19:54 a.txt
-rw-rw-r-- 1 mnntn mnntn  5 мая 20 19:52 b.txt
-rw-rw-r-- 1 mnntn mnntn  6 мая 20 19:52 c.txt
-rw-rw-r-- 1 mnntn mnntn  6 мая 20 19:54 d.txt
Scanning "./demo/client_data"
upload a.txt: ok (0.359s) Upload complete
upload d.txt: ok (0.36s) Upload complete

real    0m0,778s
user    0m0,000s
sys     0m0,003s
mnntn@dorm-server:~/networks_hw4$ █

```

Рис. 6. Инкрементальная синхронизация после изменения a.txt и добавления нового файла d.txt.

```

root@vm132052:~/networks_hw4# ls -lh demo/server_storage/smoke-001
wc -c demo/server_storage/smoke-001/a.txt demo/server_storage/smoke-001/b.txt demo/server_storage/smoke-001/c.txt demo/server_storage/smoke-001/d.txt
cksum demo/server_storage/smoke-001/a.txt demo/server_storage/smoke-001/d.txt
total 16K
-rw-r--r-- 1 root root 14 May 20 19:54 a.txt
-rw-r--r-- 1 root root  5 May 20 19:52 b.txt
-rw-r--r-- 1 root root  6 May 20 19:52 c.txt
-rw-r--r-- 1 root root  6 May 20 19:54 d.txt
14 demo/server_storage/smoke-001/a.txt
 5 demo/server_storage/smoke-001/b.txt
 6 demo/server_storage/smoke-001/c.txt
 6 demo/server_storage/smoke-001/d.txt
31 total
1444926085 14 demo/server_storage/smoke-001/a.txt
4260583175 6 demo/server_storage/smoke-001/d.txt
root@vm132052:~/networks_hw4# █

```

Рис. 7. Проверка на стороне сервера: обновленный a.txt и новый d.txt присутствуют в пользовательской директории.

Переданные файлы

Файл / набор	Размер	Тест	Комментарий
a.txt, b.txt, c.txt	5–6 В	1	Первая синхронизация малого набора файлов на пустой сервер.
a.txt, d.txt	14 В и 6 В	3	Инкрементальная синхронизация: измененный файл и новый файл.

p1.bin-p4.bin	4 × 100 MB	4 и 5	Параллельная передача четырех больших файлов без DMA и с DMA.
file_250.bin	250 MB	6	Замер времени DMA off/on для одного файла.
file_500.bin	500 MB	6	Замер времени DMA off/on для одного файла.
file_1000.bin	1000 MB	6	Замер времени DMA off/on для одного файла.

Параллельная передача файлов

Для проверки параллельной передачи был подготовлен набор из четырех файлов по 100 MB. Конфигурация клиента содержала `max_connections: 4`, после чего клиент открывал четыре независимых file-transfer соединения и отправлял все файлы одновременно.

Результаты теста без DMA:

- upload p4.bin: ok (33.746s)
- upload p2.bin: ok (34.022s)
- upload p3.bin: ok (37.944s)
- upload p1.bin: ok (42.931s)
- общее время прогона: real 0m45.242s

Результаты теста с DMA:

- upload p4.bin: ok (35.512s)
- upload p2.bin: ok (36.354s)
- upload p1.bin: ok (38.360s)
- upload p3.bin: ok (38.519s)
- общее время прогона: real 0m40.839s

Анализ `parallel_off.pcap` и `parallel_on.pcap` через `tshark -z conv,tcp` показал, что в обоих случаях наблюдалось по пять TCP-соединений:

- одно короткое управляющее соединение (control connection);
- четыре длинных файловых соединения (file transfer connections).

Для `parallel_off.pcap` длительности четырех файловых соединений составили примерно 33.75s, 34.02s, 37.94s и 42.93s. Для `parallel_on.pcap` были получены значения 35.51s, 36.35s, 38.36s и 38.52s. Это подтверждает одновременную передачу нескольких файлов и наличие отдельного канала управления.


```

mnntn@dorm-server:~/networks_hw4$ find demo/client_data -type f -delete
dd if=/dev/urandom of=demo/client_data/p1.bin bs=1M count=100 status=progress

dd if=/dev/urandom of=demo/client_data/p2.bin bs=1M count=100 status=progress

dd if=/dev/urandom of=demo/client_data/p3.bin bs=1M count=100 status=progress

dd if=/dev/urandom of=demo/client_data/p4.bin bs=1M count=100 status=progress

ls -lh demo/client_data
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0,214095 s, 490 MB/s
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0,212469 s, 494 MB/s
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0,212535 s, 493 MB/s
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0,215369 s, 487 MB/s
total 400M
-rw-rw-r-- 1 mnntn mnntn 100M мая 20 20:00 p1.bin
-rw-rw-r-- 1 mnntn mnntn 100M мая 20 20:00 p2.bin
-rw-rw-r-- 1 mnntn mnntn 100M мая 20 20:00 p3.bin
-rw-rw-r-- 1 mnntn mnntn 100M мая 20 20:00 p4.bin
mnntn@dorm-server:~/networks_hw4$

```

Рис. 8. Подготовка четырех файлов по 100 MB для теста параллельной передачи без DMA.

```

mnntn@dorm-server:~/networks_hw4$ nano config/client.yaml
mnntn@dorm-server:~/networks_hw4$ cat config/client.yaml
client_id: parallel-off-001
directory: ./demo/client_data
server_host: 103.27.157.12
server_port: 9090
max_connections: 4
dma_enabled: false
mnntn@dorm-server:~/networks_hw4$

```

Рис. 9. Изменение client_id клиента перед тестом параллельной передачи без DMA.

```
Local Server ssh, mnntn

mnntn@dorm-server:~/networks_hw4$ time ./build/mydrive_client --config
config/client.yaml sync
Scanning "./demo/client_data"

upload p3.bin: ok (35.985s) Upload complete
upload p2.bin: ok (36.698s) Upload complete
upload p1.bin: ok (37.295s) Upload complete
upload p4.bin: ok (37.679s) Upload complete

real    0m39,921s
user    0m1,783s
sys     0m0,088s
mnntn@dorm-server:~/networks_hw4$
mnntn@dorm-server:~/networks_hw4$

Local Server (1) ssh, mnntn

mnntn@dorm-server:~/networks_hw4$ sudo tcpdump -i any -w parallel_off.
pcap host 103.27.157.12 and tcp port 9090
tcpdump: data link type LINUX_SLL2
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), sna
pshot length 262144 bytes
^C2834 packets captured
5669 packets received by filter
0 packets dropped by kernel
mnntn@dorm-server:~/networks_hw4$
```

Рис. 10. Запуск передачи файлов без DMA и параллельный захват трафика tcpdump.

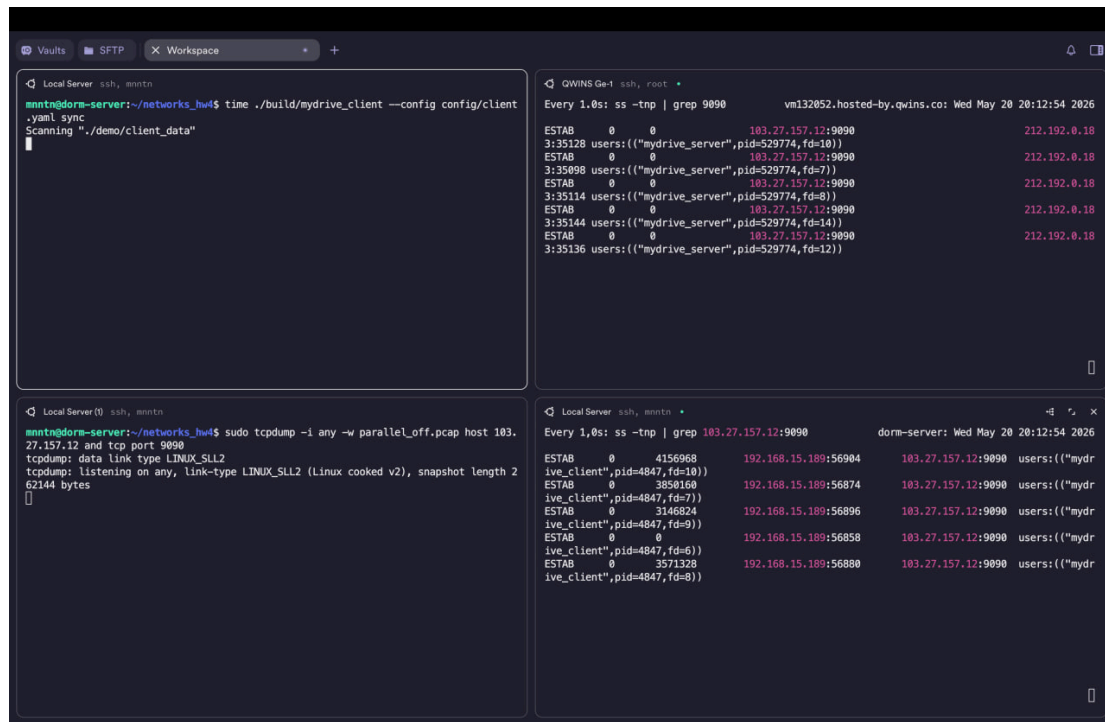


Рис. 11. Во время передачи выполнялся `ss -tnp` на клиенте и сервере; были видны несколько одновременных TCP-соединений.

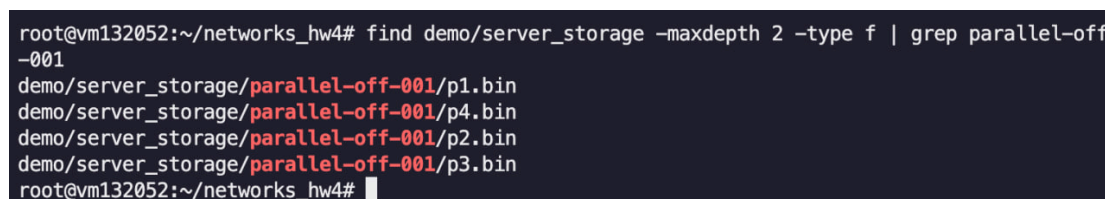


Рис. 12. Файлы `parallel-off-001` присутствуют на сервере после теста без DMA.

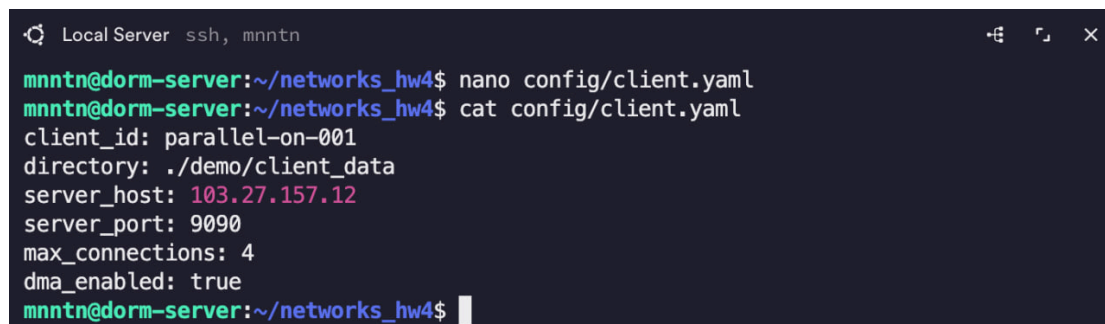


Рис. 13. Конфигурация клиента для параллельной передачи с DMA: новый `client_id`, `dma_enabled: true`.

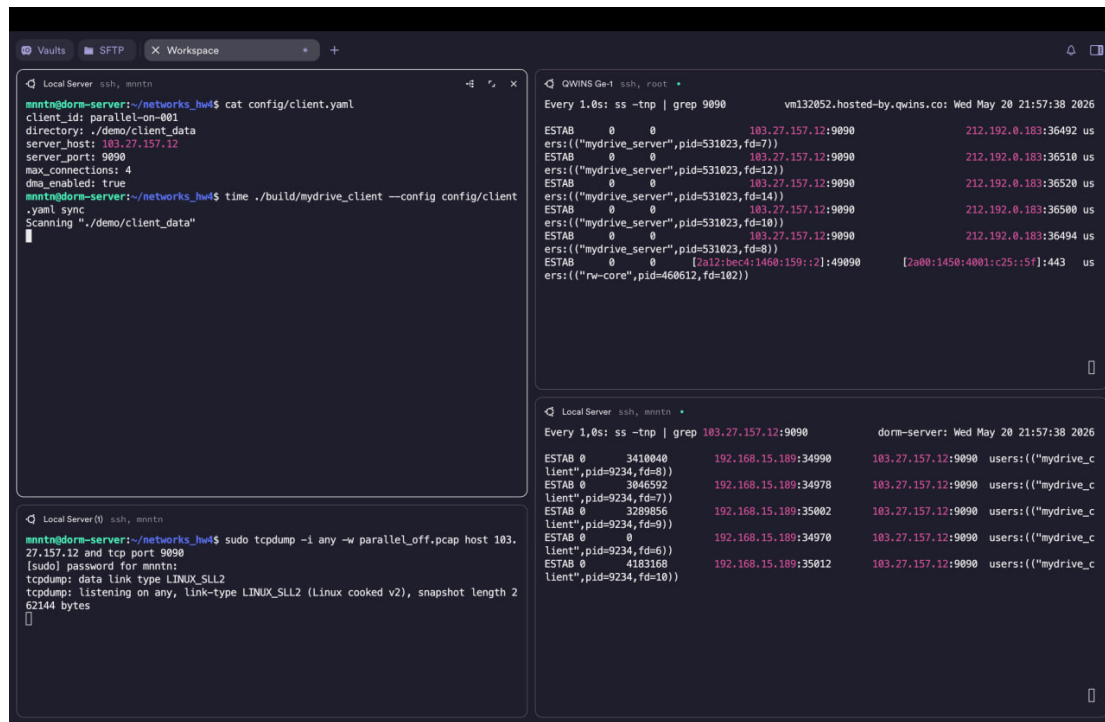


Рис. 14. Успешная параллельная передача четырех файлов при `dma_enabled: true`; одновременно наблюдаются соединения на клиенте и сервере.



Рис. 15. Файлы `parallel-on-001` присутствуют на сервере после параллельной передачи с DMA.

Наблюдения по TCP

Для анализа TCP-уровня использовались два capture-файла: `parallel_off.pcap` и `parallel_on.pcap`. Захват производился на Linux-клиенте через `tcpdump`, а дальнейший разбор выполнялся через Wireshark и `tshark`.

MSS и Window

В SYN/SYN-ACK пакетах обоих capture-наборов наблюдались одинаковые TCP options:

- MSS = 65495
- SACK permitted
- Window Scale = 7 (множитель 128)
- TCP timestamps

В начале соединения сервер присылал `win 65483` в SYN/ACK, а затем в ACK-пакетах по мере передачи наблюдались значения окна 512, 848, 792, 692 и другие. Внутри одного потока окно уменьшалось и изменялось динамически в зависимости от состояния приемной стороны.

Slow start

На уровне packet trace наблюдалось постепенное наращивание обмена данными после установления соединения: сначала управляющее соединение, затем почти одновременно четыре файловых соединения, после чего начинался интенсивный обмен ACK и data packets. Для более формального

анализа slow start в финальной версии отчета следует приложить скрин одного из TCP Stream Graphs из Wireshark.

Congestion

В разборе `parallel_off.pcap` при чтении первых пакетов были видны ACK-пакеты с `sack-опциями` и временами нулевым окном (`win 0`), что указывает на временную перегрузку приемной стороны или необходимость дожидаться освобождения буфера. Это не привело к срыву передачи: все четыре файла были успешно переданы и подтверждены сервером. Для режима DMA после исправления клиентской ветки `sendfile` передача также завершалась успешно.

Для финальной версии отчета рекомендуется добавить из Wireshark следующие скриншоты:

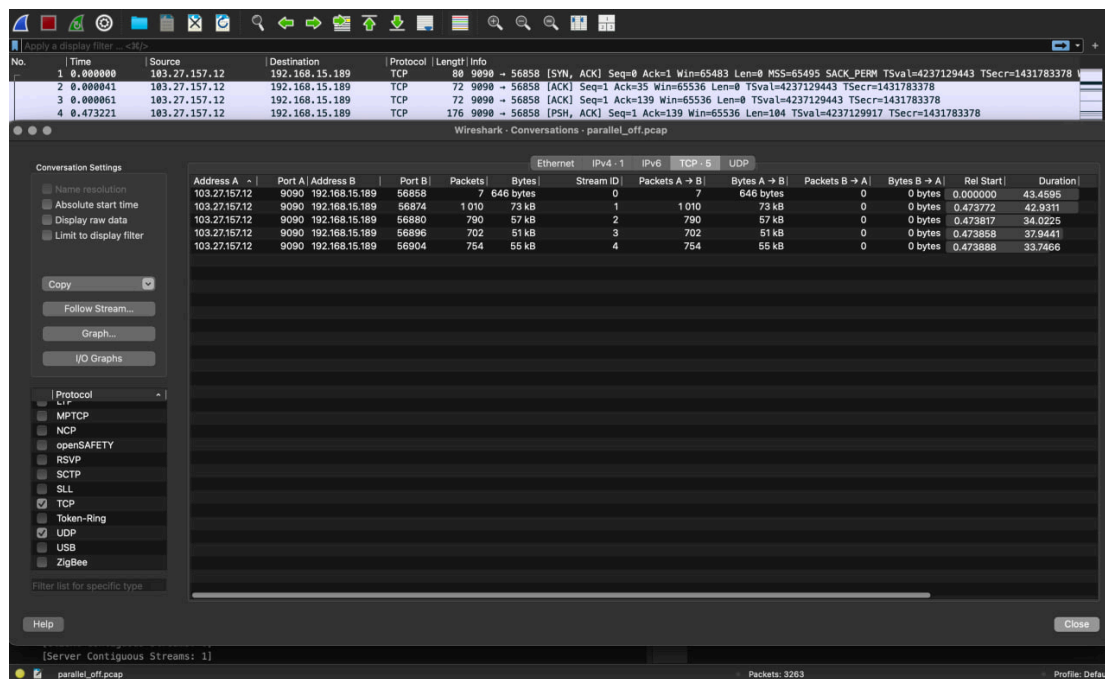


Рис. 16. Окно Statistics -> Conversations -> TCP для `parallel_off.pcap`: одно короткое управляющее соединение и четыре длительных файловых соединения.

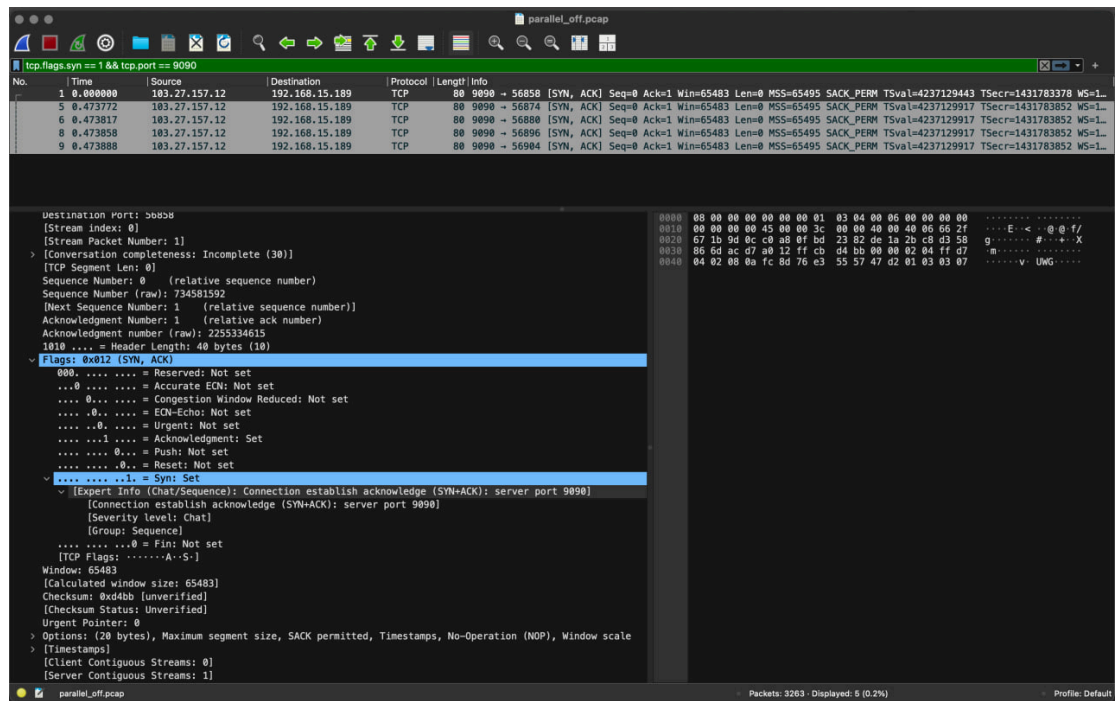


Рис. 17. TCP handshake для одного из соединений передачи файла в parallel_off.pcap.

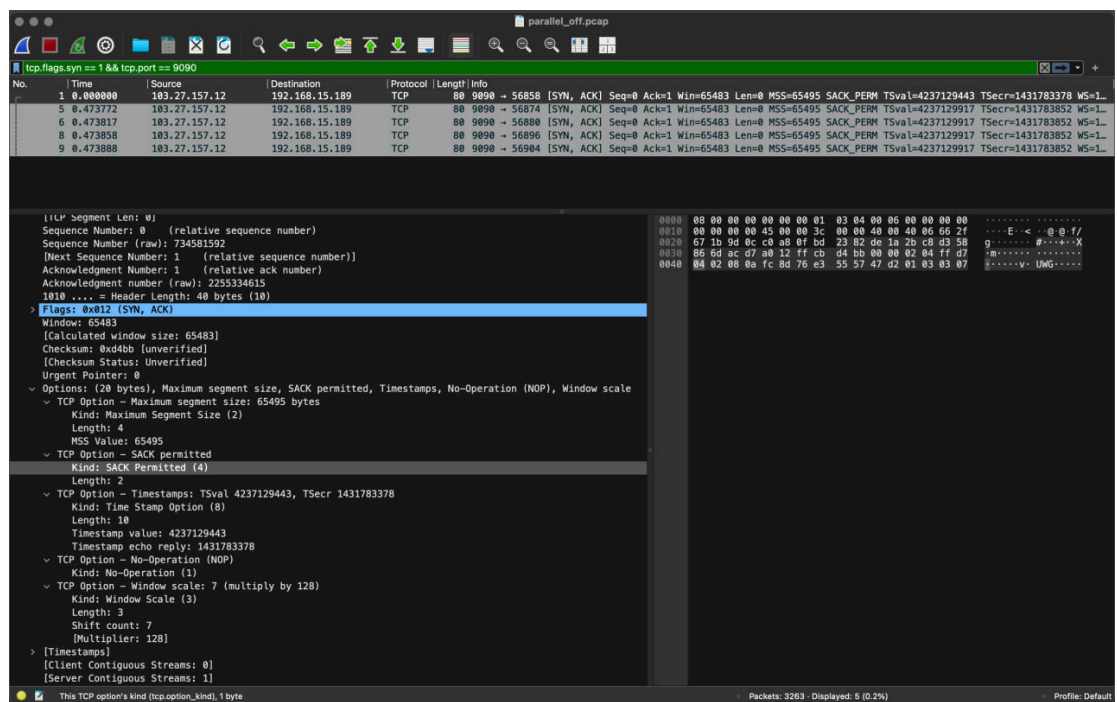


Рис. 18. TCP options в SYN, ACK пакете: MSS, SACK permitted, timestamps, Window Scale, а также начальное окно приема.

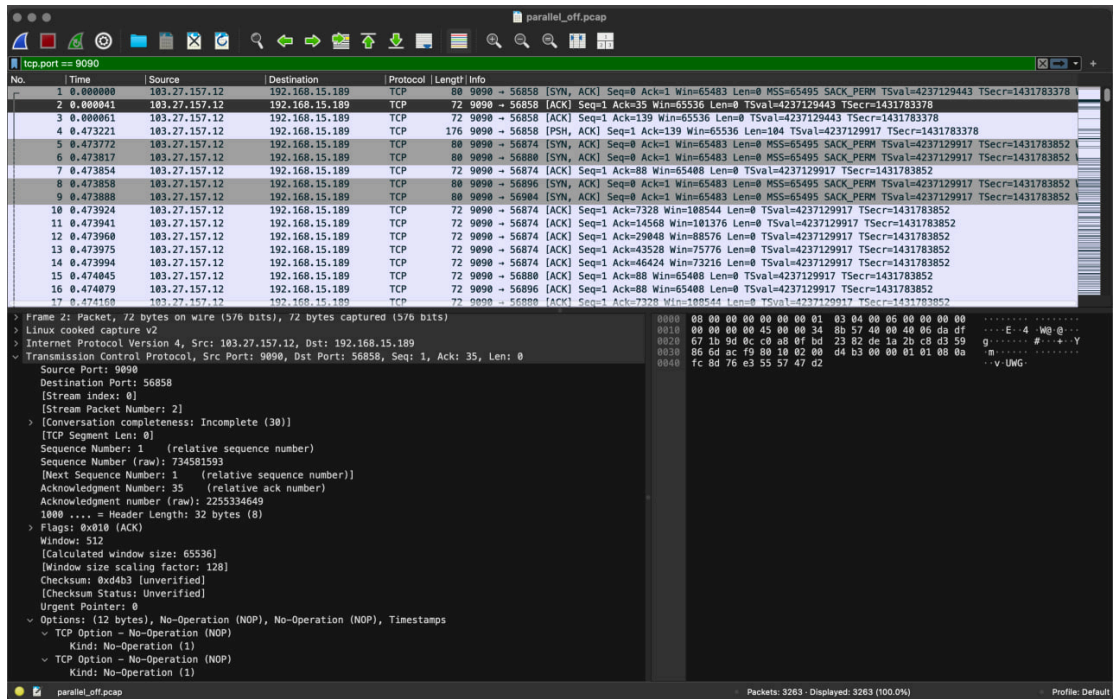


Рис. 19. Пример обычного пакета передачи данных с Window size value и Calculated window size.

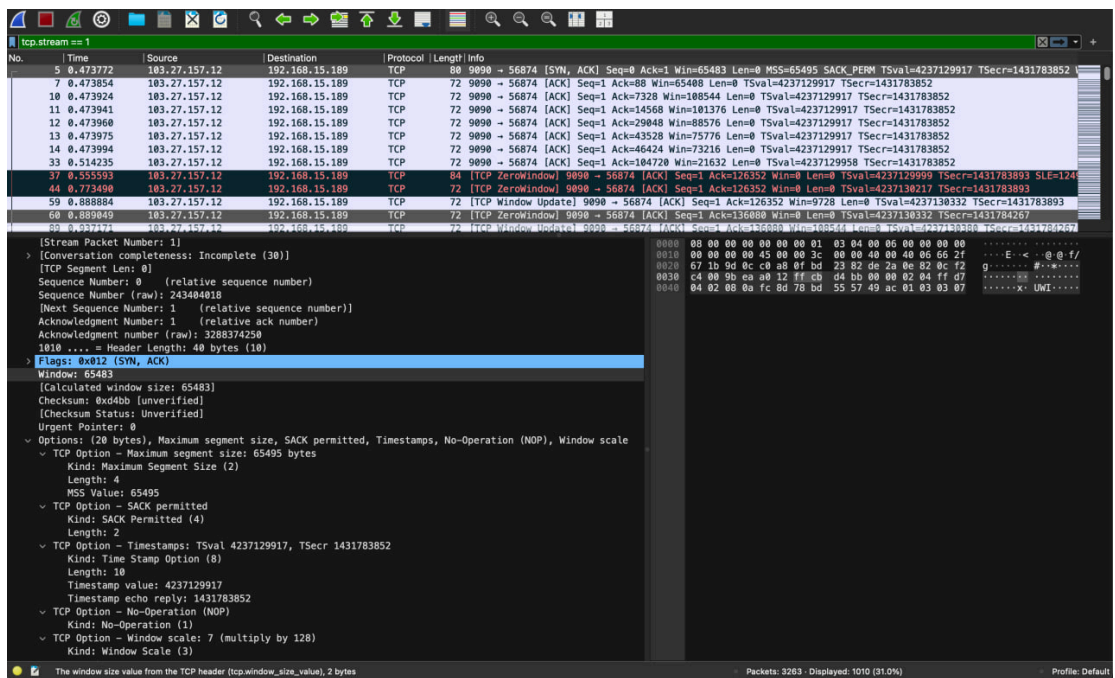


Рис. 20. Один активный tcp.stream при передаче большого файла.

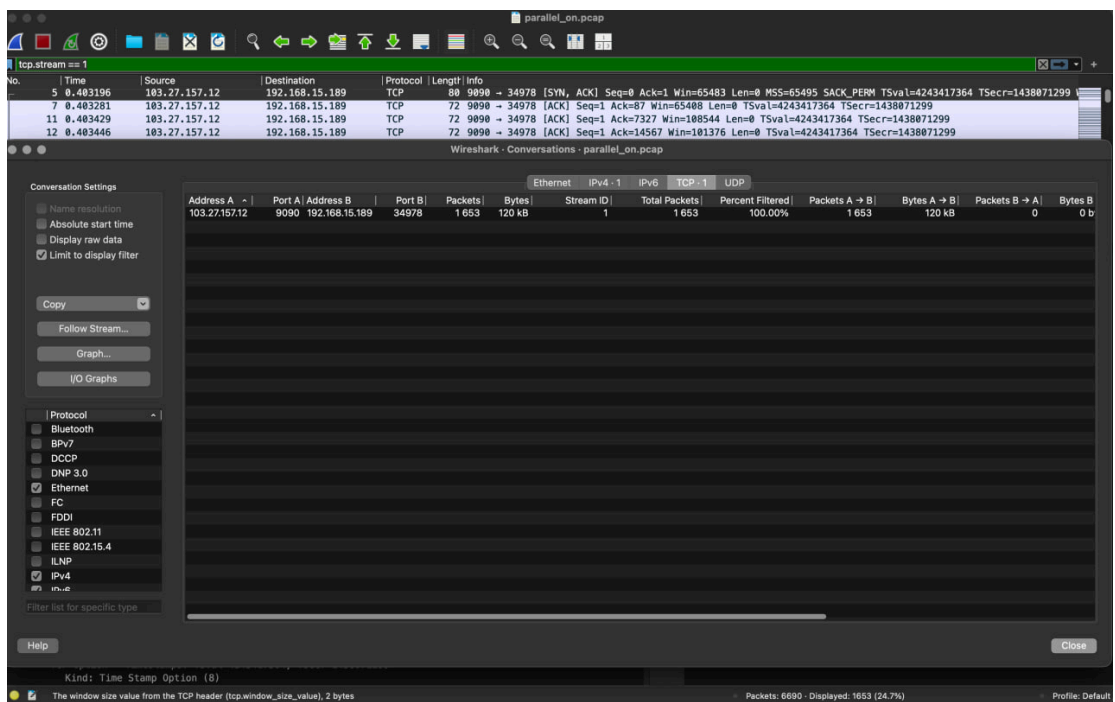


Рис. 21. Окно Statistics -> Conversations -> TCP для parallel_on.pcap: параллельная передача в режиме DMA.

DMA и сравнение с обычной передачей

В реализации предусмотрены два режима работы клиента:

- dma_enabled: false — обычная буферизованная передача файла;
- dma_enabled: true — zero-copy путь на Linux через sendfile.

Использование sendfile было дополнительно подтверждено через strace -f -e trace=sendfile: в режиме DMA он наблюдались реальные системные вызовы sendfile(...), а также ситуации EAGAIN, корректно обрабатываемые клиентом при работе по неблокирующему сокету.

Таблица замеров

Размер	Соединения	DMA	Upload	Общее real	Комментарий
250 MB	1 файл	off	25.503s	27.272s	Базовый прогон без DMA.
250 MB	1 файл	on	24.509s	26.072s	Небольшое ускорение при DMA.
500 MB	1 файл	off	51.568s	54.341s	Передача одного файла без DMA.
500 MB	1 файл	on	48.905s	51.716s	Ускорение примерно на 2.6 секунды.
1000 MB	1 файл	off	97.459s	102.507s	Передача одного файла размером 1 GB без DMA.
1000 MB	1 файл	on	99.652s	104.720s	В этом прогоне DMA оказал-

					ся чуть медленнее.
--	--	--	--	--	--------------------

Графики

По полученным замерам нельзя сделать вывод о гарантированном ускорении во всех сценариях. Для файлов 250 MB и 500 MB режим DMA on показал умеренное улучшение, однако на 1000 MB в конкретном прогоне время оказалось немного хуже, чем у DMA off. Это может объясняться сетевой вариативностью, работой дисковой подсистемы сервера и тем фактом, что до начала передачи в обоих режимах выполняется полный расчет checksum по файлу.

```
mnntn@dorm-server:~/networks_hw4$ find demo/client_data -type f -delete
dd if=/dev/urandom of=demo/client_data/file_250.bin bs=1M count=250 status=progress
250+0 records in
250+0 records out
262144000 bytes (262 MB, 250 MiB) copied, 0,534562 s, 490 MB/s
mnntn@dorm-server:~/networks_hw4$ nano config/client.yaml
mnntn@dorm-server:~/networks_hw4$ cat config/client.yaml
client_id: dma-off-250
directory: ./demo/client_data
server_host: 103.27.157.12
server_port: 9090
max_connections: 4
dma_enabled: false
mnntn@dorm-server:~/networks_hw4$ time ./build/mydrive_client --config config/client
.yaml sync
Scanning "./demo/client_data"
upload file_250.bin: ok (25.503s) Upload complete

real    0m27,272s
user    0m1,160s
sys     0m0,069s
mnntn@dorm-server:~/networks_hw4$
```

Рис. 22. Замер времени для file_250.bin в режиме DMA off.

```
mnntn@dorm-server:~/networks_hw4$ time ./build/mydrive_client --config config/client
.yaml sync
Scanning "./demo/client_data"
upload file_250.bin: ok (24.509s) Upload complete

real    0m26,072s
user    0m1,145s
sys     0m0,040s
mnntn@dorm-server:~/networks_hw4$ cat config/client.yaml
client_id: dma-on-250
directory: ./demo/client_data
server_host: 103.27.157.12
server_port: 9090
max_connections: 4
dma_enabled: true
```

Рис. 23. Замер времени для file_250.bin в режиме DMA on.

```

mnntn@dorm-server:~/networks_hw4$ find demo/client_data -type f -delete
mnntn@dorm-server:~/networks_hw4$ dd if=/dev/urandom of=demo/client_data/file_500.bin
bs=1M count=500 status=progress
492830720 bytes (493 MB, 470 MiB) copied, 1 s, 492 MB/s
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 1,0654 s, 492 MB/s
mnntn@dorm-server:~/networks_hw4$ nano config/client.yaml
mnntn@dorm-server:~/networks_hw4$ cat config/client.yaml
client_id: dma-off-500
directory: ./demo/client_data
server_host: 103.27.157.12
server_port: 9090
max_connections: 4
dma_enabled: false
mnntn@dorm-server:~/networks_hw4$ time ./build/mydrive_client --config config/client
.yaml sync
Scanning "./demo/client_data"
upload file_500.bin: ok (51.568s) Upload complete

real    0m54,341s
user    0m2,336s
sys     0m0,127s
mnntn@dorm-server:~/networks_hw4$

```

Рис. 24. Замер времени для file_500.bin в режиме DMA off.

```

mnntn@dorm-server:~/networks_hw4$ nano config/client.yaml
mnntn@dorm-server:~/networks_hw4$ cat config/client.yaml
client_id: dma-on-500
directory: ./demo/client_data
server_host: 103.27.157.12
server_port: 9090
max_connections: 4
dma_enabled: true
mnntn@dorm-server:~/networks_hw4$ time ./build/mydrive_client --config config/client
.yaml sync
Scanning "./demo/client_data"
upload file_500.bin: ok (48.905s) Upload complete

real    0m51,716s
user    0m2,289s
sys     0m0,075s
mnntn@dorm-server:~/networks_hw4$

```

Рис. 25. Замер времени для file_500.bin в режиме DMA on.

```

mnntn@dorm-server:~/networks_hw4$ find demo/client_data -type f -delete
dd if=/dev/urandom of=demo/client_data/file_1000.bin bs=1M count=1000 status=progress
985661440 bytes (986 MB, 940 MiB) copied, 2 s, 493 MB/s
1000+0 records in
1000+0 records out
1048576000 bytes (1,0 GB, 1000 MiB) copied, 2,12878 s, 493 MB/s
mnntn@dorm-server:~/networks_hw4$ nano config/client.yaml
mnntn@dorm-server:~/networks_hw4$ cat config/client.yaml
client_id: dma-off-1000
directory: ./demo/client_data
server_host: 103.27.157.12
server_port: 9090
max_connections: 4
dma_enabled: false
mnntn@dorm-server:~/networks_hw4$ time ./build/mydrive_client --config config/client
.yaml sync
Scanning "./demo/client_data"
upload file_1000.bin: ok (97.459s) Upload complete

real    1m42,507s
user    0m4,623s
sys     0m0,247s
mnntn@dorm-server:~/networks_hw4$

```

Рис. 26. Замер времени для file_1000.bin в режиме DMA off.

```

mnntn@dorm-server:~/networks_hw4$ nano config/client.yaml
mnntn@dorm-server:~/networks_hw4$ cat config/client.yaml
client_id: dma-on-1000
directory: ./demo/client_data
server_host: 103.27.157.12
server_port: 9090
max_connections: 4
dma_enabled: true
mnntn@dorm-server:~/networks_hw4$ time ./build/mydrive_client --config config/client
.yaml sync
Scanning "./demo/client_data"
upload file_1000.bin: ok (99.652s) Upload complete

real    1m44,720s
user    0m4,504s
sys     0m0,128s
mnntn@dorm-server:~/networks_hw4$

```

Рис. 27. Замер времени для file_1000.bin в режиме DMA on.

Вывод

В рамках работы было реализовано многопользовательское клиент-серверное приложение для синхронизации файловой директории по TCP. Решение включает собственный бинарный протокол, асинхронную обработку соединений, отдельные control/file соединения, параллельную передачу нескольких файлов и режим zero-copy передачи на Linux через sendfile.

Эксперименты подтвердили:

- корректную первичную синхронизацию файлов;
- отсутствие лишней передачи при повторной синхронизации без изменений;
- корректную инкрементальную синхронизацию измененного и нового файлов;
- параллельную передачу четырех больших файлов по четырем независимым TCP-соединениям;

- работоспособность DMA-режима на Linux и реальное использование `sendfile`, подтвержденное через `strace`.

Сравнение DMA off/on показало, что ускорение не является гарантированным во всех тестах, однако на части прогонов режим DMA давал умеренное уменьшение времени передачи. Это согласуется с тем, что итоговое время зависит не только от копирования данных в сокет, но и от расчета checksum, состояния сети и записи файла на сервере.